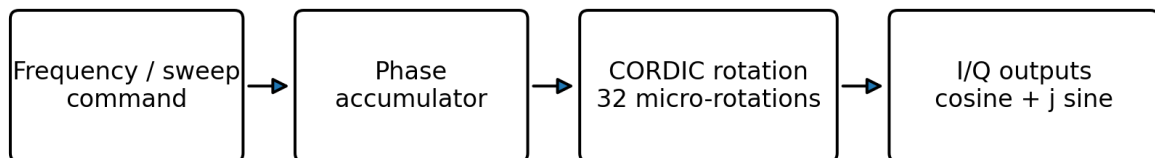


CORDIC ALGORITHM

Efficient generation of fixed tones and linear frequency-modulated signals

CORDIC signal-generation chain



Only shifts, additions/subtractions, and a small arctangent table

Rotation-mode implementation based on shifts, additions and a compact arctangent look-up table.

Institution

Universidad Politécnica de Madrid

Programme

Telecommunications Engineering

Student

Guillermo Aladro

Date

July 2026

DIGITAL SIGNAL PROCESSING LABORATORY

Contents

1. Introduction.....	3
2. Objectives and deliverables.....	3
3. CORDIC theoretical basis.....	4
3.1 Rotation equations.....	4
3.2 Number of iterations and precision.....	4
3.3 Gain compensation and full-circle preprocessing.....	6
3.4 Fixed-point input emulation.....	6
4. MATLAB implementation.....	6
4.1 mcordic_r2.m.....	6
4.2 CORDIC.m.....	7
4.3 Toolbox-independent STFT.....	7
5. Fixed-tone generation.....	7
6. Repeated LFM generation.....	9
7. Numerical validation.....	11
8. Hardware-oriented considerations.....	11
9. Conclusions.....	12
References.....	12
Appendix A. Complete MATLAB code.....	13
Appendix B. Execution instructions.....	18

Package contents

- CORDIC_Report.pdf - this English technical report.
- code/CORDIC.m - main experiment script for tones, LFM, FFTs, STFT and validation.
- code/mcordic_r2.m - full-circle CORDIC rotation function.
- results_figures/ - reference plots obtained from an equivalent numerical implementation.

Notation

Symbol	Meaning	Value used
f_s	Sampling frequency	44 kHz
N	Number of samples	44,000 for each tone; 88,000 for LFM
M	CORDIC iterations	32
K_M	CORDIC gain	1.6467602581
A	Output amplitude	0.5

1. Introduction

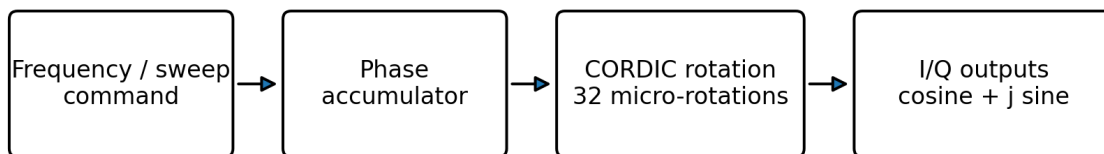
The Coordinate Rotation Digital Computer (CORDIC) algorithm evaluates trigonometric functions through a sequence of elementary vector rotations. Its main hardware advantage is that every micro-rotation can be implemented with additions, subtractions, sign decisions and binary shifts. No general-purpose multiplier is required in the iterative core, which makes the method suitable for FPGA, ASIC and resource-constrained digital signal-processing architectures.

This practice implements CORDIC in rotation mode and uses it as a numerically controlled oscillator. A phase accumulator supplies the requested angle, and the CORDIC block returns the in-phase and quadrature components of the corresponding complex exponential. The same structure is then used for three fixed tones and for a repeated linear frequency-modulated (LFM) sweep.

2. Objectives and deliverables

- Implement a full-circle rotation-mode CORDIC function in MATLAB.
- Compensate the intrinsic CORDIC gain and emulate the intended fixed-point input preparation.
- Generate 440 Hz, 1 kHz and 2.3 kHz complex tones at a 44 kHz sampling rate.
- Generate a two-second LFM waveform that repeatedly sweeps from 500 Hz to 3 kHz every second.
- Verify the outputs in the time, frequency and time-frequency domains.
- Measure the numerical error against MATLAB sine and cosine references.

CORDIC signal-generation chain



Only shifts, additions/subtractions, and a small arctangent table

Figure 1. Signal-generation architecture used in the practice.

3. CORDIC theoretical basis

3.1 Rotation equations

A two-dimensional rotation by an angle θ can be written as a matrix product. CORDIC decomposes that rotation into a sum of elementary angles $\text{atan}(2^{-i})$. Because the tangent of every elementary angle is a power of two, the multiplication by 2^{-i} becomes a binary right shift in fixed-point hardware.

$$x_{i+1} = x_i - d_i y_i 2^{-i}$$

$$y_{i+1} = y_i + d_i x_i 2^{-i}$$

$$z_{i+1} = z_i - d_i \text{atan}(2^{-i})$$

The residual angle z_i determines the rotation direction. The code selects $d_i = +1$ when z_i is non-negative and $d_i = -1$ otherwise. With each iteration, the absolute residual angle decreases and the vector approaches the requested direction.

One CORDIC micro-rotation

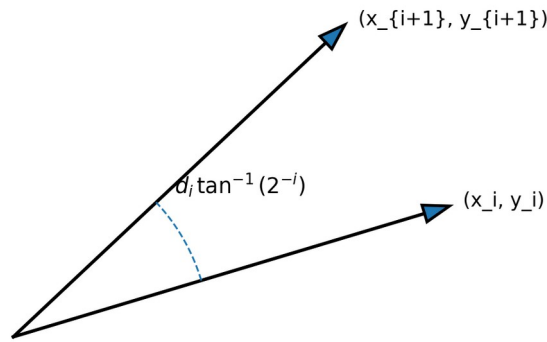


Figure 2. Geometrical interpretation of a single micro-rotation.

3.2 Number of iterations and precision

The angle table contains 32 entries, from $\text{atan}(1)$ down to $\text{atan}(2^{-31})$. In an ideal fixed-point implementation, each additional useful iteration contributes approximately one extra bit of angular precision. The final error is also affected by word length, rounding, saturation and the precision of the stored angle table. In this floating-point MATLAB model, the 32-step iterative approximation is the dominant algorithmic mechanism while the input amplitude is explicitly quantized to 21 fractional bits.

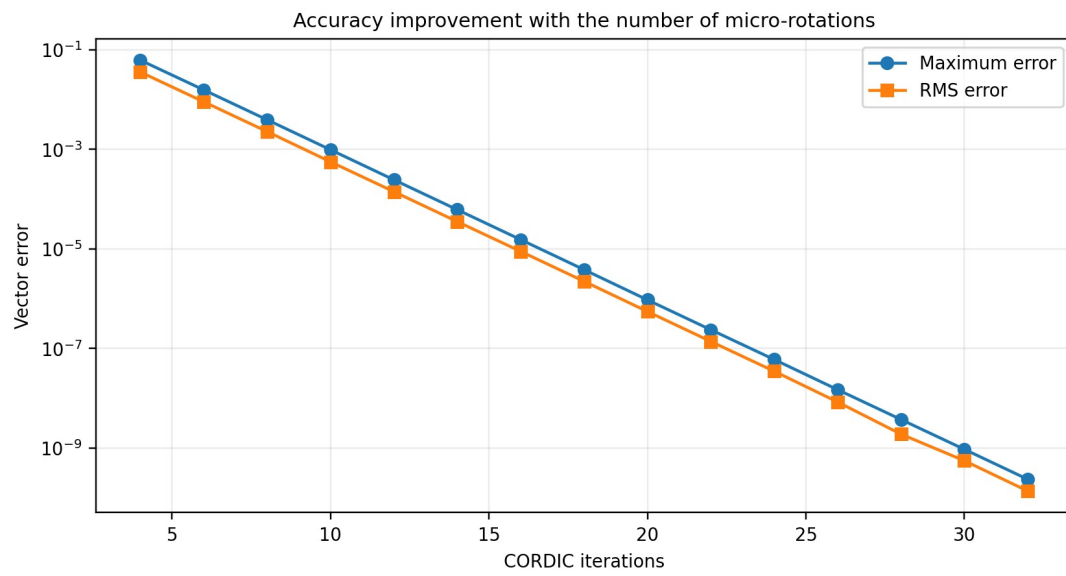


Figure 3. Reduction of numerical error as the iteration count increases.

3.3 Gain compensation and full-circle preprocessing

The elementary matrices used by CORDIC introduce a deterministic scale factor. For M iterations the accumulated gain is:

$$K_M = \text{product from } i=0 \text{ to } M-1 \text{ of } \sqrt{1 + 2^{-2i}}$$

For 32 iterations, the evaluated gain is $K_{32} = 1.6467602581$. Therefore, the initial vector is divided by this value. With a requested amplitude of 0.5, the pre-scaled x input is 0.3036264675. After the iterative rotations, the output magnitude is approximately 0.5.

The standard circular CORDIC convergence interval is narrower than the complete $[-\pi, \pi]$ range. The function therefore performs an exact initial $\pm\pi/2$ rotation by swapping the x and y coordinates and changing signs. The remaining angle is then inside the convergence interval of the iterative stage. This preprocessing avoids quadrant-specific sine/cosine logic in the calling script.

3.4 Fixed-point input emulation

The original practice describes a 1T31-style representation. The MATLAB statement first truncates the amplitude to 21 fractional bits and then maps that value into a 31-fractional-bit scale by multiplying by 2^{10} . Algebraically, the operation is equivalent to $\text{floor}(x \cdot 2^{21}) / 2^{21}$. The quantized value is subsequently divided by the CORDIC gain. The chosen amplitude 0.5 is exactly representable, so this particular test does not introduce input quantization error.

```
x_quantized = (floor(xin * 2^21) * 2^10) / 2^31;
x_actual = x_quantized / cordic_gain;
```

4. MATLAB implementation

The corrected practice consists of two executable MATLAB source files. The function file contains the reusable rotation kernel, while the main script defines experiment parameters, generates signals, calculates spectra, builds a toolbox-independent STFT and saves the numerical results.

4.1 mcordic_r2.m

```
function [xout, yout] = mcordic_r2(xin, yin, angle_table, angle)
    z = angle;
    d = 1; if z < 0, d = -1; end
    y = xin * d;
    x = -yin * d;
    z = z - d*pi/2;

    for k = 1:length(angle_table)
        d = 1; if z < 0, d = -1; end
        shift = 2^(-(k - 1));
        x_next = x - d*y*shift;
        y_next = y + d*x*shift;
        z_next = z - d*angle_table(k);
        x = x_next; y = y_next; z = z_next;
    end
    xout = x; yout = y;
end
```

Temporary variables are essential because $x_{(i+1)}$ and $y_{(i+1)}$ must both be computed from the previous iteration values. Updating x before calculating y would change the recurrence and produce an incorrect rotation.

4.2 CORDIC.m

The main script precomputes the arctangent table and CORDIC gain once. For each generated sample, a phase accumulator is updated and wrapped to $[-\pi, \pi]$. The phase is passed to `mcordic_r2` together with the fixed pre-scaled input vector. The output pair forms the analytic signal $x[n] + j y[n]$.

```
phase_increment = 2*pi*frequency/fs;
for n = 1:N_tone
    [x_rotated, y_rotated] = mcordic_r2( ...
        x_actual, y_actual, angle_table, phase);
    x_tones(tone_index,n) = x_rotated;
    y_tones(tone_index,n) = y_rotated;
    phase = phase + phase_increment;
    if phase > pi
        phase = phase - 2*pi;
    elseif phase < -pi
        phase = phase + 2*pi;
    end
end
```

The revised script corrects the non-MATLAB Spanish pseudocode tokens in the supplied draft, including `claro`, `para`, `si`, `fin`, `trazar`, `redondear` and `piso`. It also standardizes variable names, fixes inconsistent references such as `f_start/f_inicio` and `x_out/x_salida`, and implements all three requested fixed tones rather than only the 440 Hz case.

4.3 Toolbox-independent STFT

To keep the practice portable, the spectrogram is calculated explicitly. The LFM signal is divided into 1024-sample Hann-windowed frames with 512-sample overlap. Each frame is transformed with a 1024-point FFT, and only the non-negative frequency bins are retained. Consequently, the code does not require the Signal Processing Toolbox.

5. Fixed-tone generation

Three one-second signals are generated at 44 kHz. Since each requested frequency is an integer number of hertz and the observation interval is exactly one second, every tone completes an integer number of cycles. This places the spectral component exactly on a DFT bin and minimizes leakage in the ideal model.

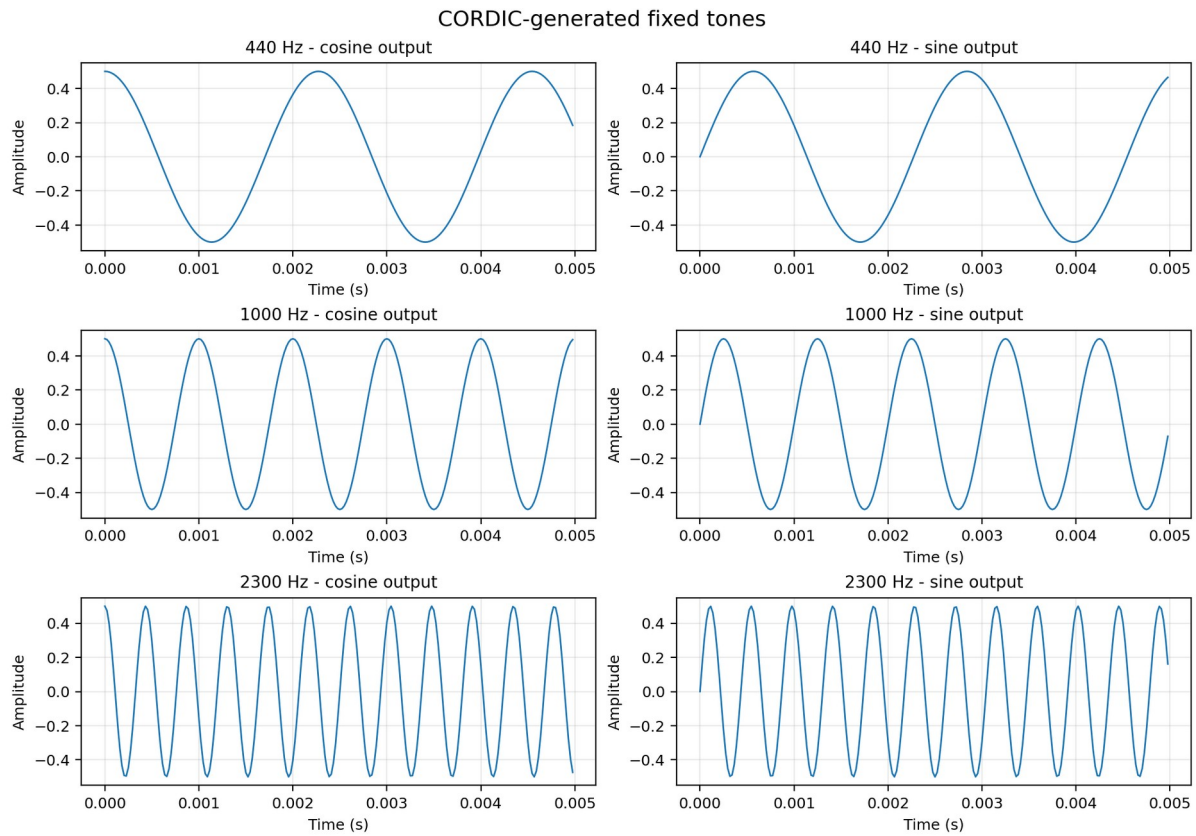


Figure 4. First 5 ms of the cosine and sine outputs for the three generated tones.

5.1 Time-domain observations

The two outputs have equal amplitudes and are in quadrature. The cosine component starts close to +0.5 at zero phase, while the sine component starts close to zero. As the frequency increases, the number of periods visible in the same five-millisecond interval increases accordingly. The amplitude remains constant because the same gain-compensated input vector is used for all phases.

5.2 Frequency-domain observations

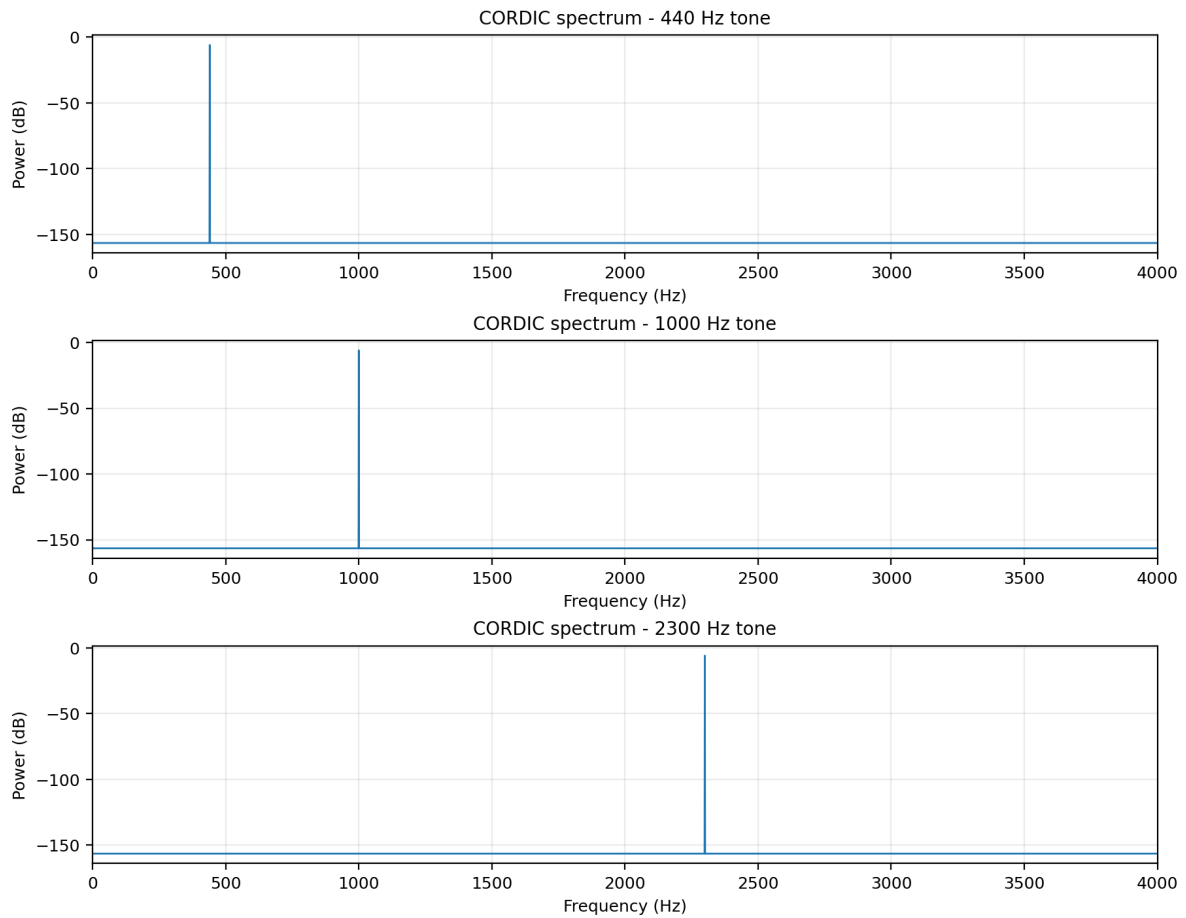


Figure 5. One-sided power spectra of the analytic CORDIC tones.

The FFT maxima are measured at 440 Hz, 1000 Hz and 2300 Hz, exactly matching the requested values. The complex representation concentrates the desired rotating phasor at the positive-frequency bin. The very low remaining floor is caused by finite numerical precision and plotting in decibels.

Requested frequency	Measured FFT peak	Duration	Amplitude
440 Hz	440 Hz	1 s	0.5
1000 Hz	1000 Hz	1 s	0.5
2300 Hz	2300 Hz	1 s	0.5

6. Repeated LFM generation

The LFM section generates a two-second signal. During each one-second ramp, the instantaneous frequency increases linearly from 500 Hz to 3000 Hz. At the end of the ramp, the frequency command returns to 500 Hz and the sweep repeats. The phase accumulator is not reset, so the complex signal remains phase-continuous apart from the unavoidable change in instantaneous-frequency slope.

$$f[k] = 500 + \text{mod}(k, 44000) * (3000 - 500) / (44000 - 1)$$

$$\text{phi}[k] = \text{wrap}_{(-\pi, \pi)} \{ \text{phi}[k-1] + 2 \pi f[k] / f_s \}$$

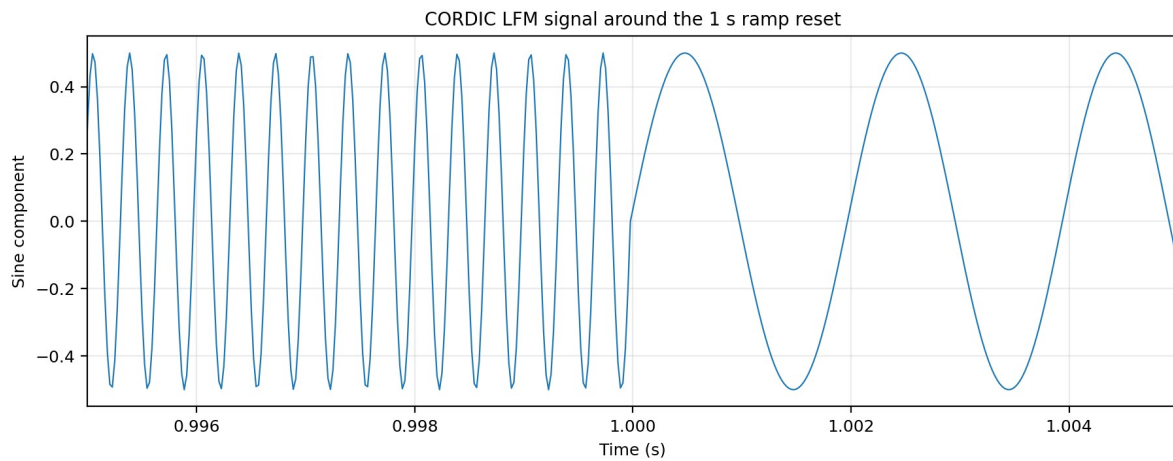


Figure 6. Time-domain detail around the frequency-ramp reset at one second.

6.1 LFM spectrum

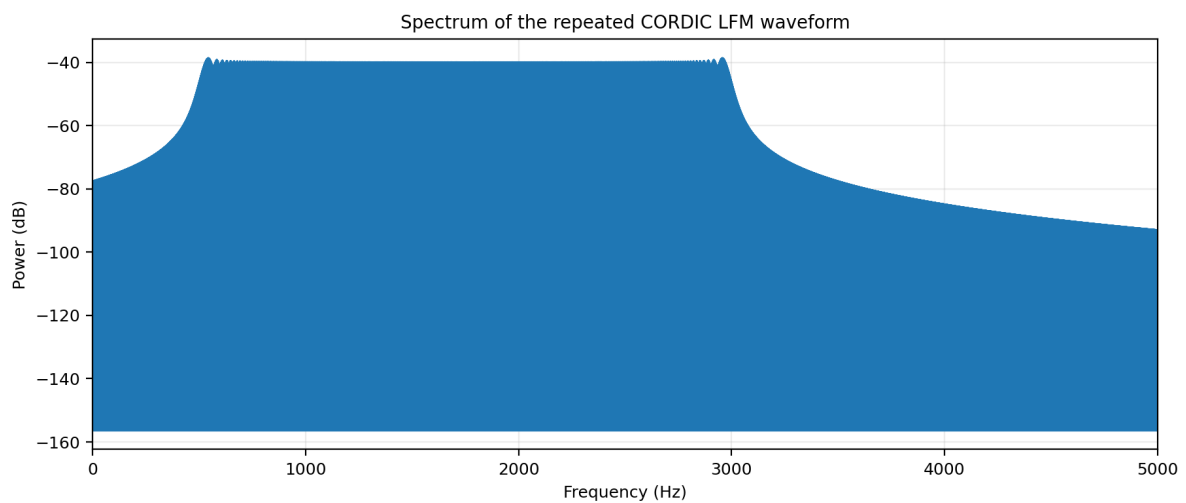


Figure 7. Overall spectrum of the two repeated sweeps.

Unlike a fixed tone, the LFM waveform distributes energy across its swept band. The dominant spectral region extends from approximately 500 Hz to 3 kHz. The finite observation time, the frequency reset and the window implicit in the two-second record produce sidelobes and ripple outside the ideal rectangular band.

6.2 Time-frequency analysis

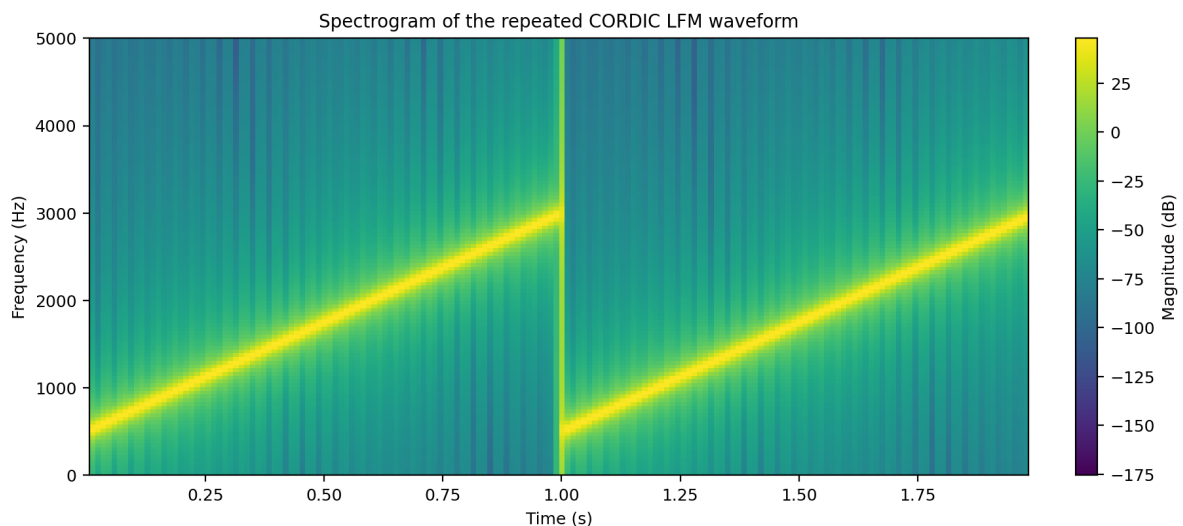


Figure 8. STFT of the repeated LFM signal.

The spectrogram provides the clearest validation of the modulation law. Two rising ridges are visible: each begins near 500 Hz, reaches 3 kHz after one second, and then restarts at 500 Hz. The ridge thickness is governed by the 1024-sample analysis window and represents the usual time-frequency resolution trade-off.

7. Numerical validation

The CORDIC output was evaluated for 2001 uniformly spaced angles over the complete $[-\pi, \pi]$ interval and compared with the references $0.5 \cos(\theta)$ and $0.5 \sin(\theta)$. The maximum Euclidean vector error was $2.327\text{e-}10$, while the RMS vector error was $1.352\text{e-}10$. These results confirm correct quadrant handling, gain compensation and iteration indexing.

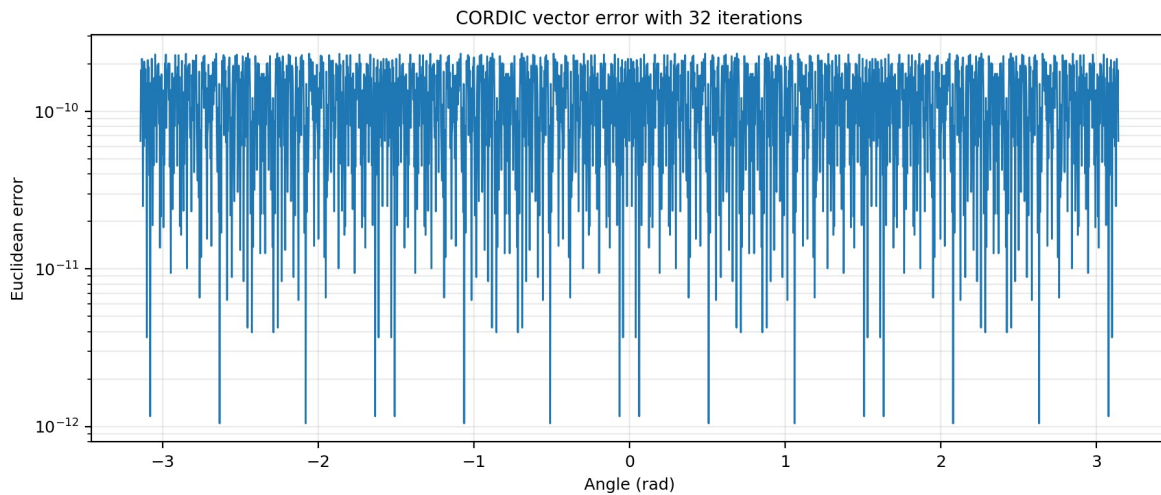


Figure 9. Vector error over the complete input-angle interval.

7.1 Main verification checks

Check	Result
Magnitude preservation	$\sqrt{x^2+y^2}$ remains approximately 0.5 after gain compensation.
Quadrature relation	The sine and cosine channels are separated by 90 degrees.
Frequency accuracy	All fixed-tone FFT peaks coincide with their requested DFT bins.
Full-circle operation	No discontinuity or incorrect sign appears at quadrant boundaries.
LFM trajectory	The STFT ridge follows the specified 500-3000 Hz repeated ramp.

8. Hardware-oriented considerations

The MATLAB model uses floating-point values to demonstrate the algorithm, but the recurrence is directly compatible with fixed-point hardware. At 44,000 output samples per second and 32 iterations per sample, a fully iterative design performs 1.408 million micro-rotations per second. Each micro-rotation requires two data-path additions/subtractions, one residual-angle subtraction and wiring shifts. The implementation can therefore be serialized with one reused stage, partially unrolled, or fully pipelined depending on throughput and area requirements.

- A serial architecture minimizes resources but needs at least 32 clock cycles per output sample.

- A fully pipelined architecture can produce one sample per clock after the initial latency.
- The angle table can be stored as constants or in a small ROM.
- Word growth, rounding and saturation must be defined explicitly for a synthesizable fixed-point version.
- The initial $\pm\pi/2$ rotation requires only swaps and sign changes, not a multiplier.

9. Conclusions

The completed practice demonstrates that a single CORDIC rotation kernel can serve as the trigonometric engine of both a fixed-frequency oscillator and an LFM generator. The corrected MATLAB implementation covers the entire circle, compensates the CORDIC gain, generates the three requested tones, produces a repeated 500-3000 Hz chirp, and validates the results through FFT, STFT and direct numerical comparison.

With 32 micro-rotations, the floating-point reference model reaches sub-nanounit vector error for the tested amplitude. More importantly from a digital-design perspective, the iterative core retains the defining CORDIC property: the variable multiplications are replaced by power-of-two shifts. The code therefore provides a clear starting point for a later fixed-point or HDL implementation.

References

1. J. E. Volder, "The CORDIC Trigonometric Computing Technique," IRE Transactions on Electronic Computers, 1959.
2. J. S. Walther, "A Unified Algorithm for Elementary Functions," Spring Joint Computer Conference, 1971.
3. MATLAB source code supplied for the practice and corrected in this package.

Appendix A. Complete MATLAB code

A.1 mcordic_r2.m

```
function [xout, yout] = mcordic_r2(xin, yin, angle_table, angle)
%MCORDIC_R2 CORDIC rotation mode with full-circle angle support.
%
% [XOUT, YOUT] = MCORDIC_R2(XIN, YIN, ANGLE_TABLE, ANGLE)
% rotates the input vector [XIN, YIN] by ANGLE radians using only
% additions, subtractions, and powers-of-two scale factors.
%
% ANGLE_TABLE must contain atan(2^(-i)) for i = 0, 1, ..., M-1.
% The caller is responsible for compensating the CORDIC gain in the
% input vector when a unit-amplitude rotation is required.
%
% The initial +/-pi/2 rotation extends the convergence interval so that
% input angles in [-pi, pi] can be processed directly.

z = angle;

% Direction for the initial +/-pi/2 rotation.
if z >= 0
    d = 1;
else
    d = -1;
end

% Initial exact rotation by d*pi/2:
% [x; y] <- R(d*pi/2) * [xin; yin]
y = xin * d;
x = -yin * d;
z = z - d * pi / 2;

% Iterative CORDIC micro-rotations.
for k = 1:length(angle_table)
    if z >= 0
        d = 1;
    else
        d = -1;
    end

    shift = 2^(-(k - 1));
    x_next = x - d * y * shift;
    y_next = y + d * x * shift;
    z_next = z - d * angle_table(k);

    x = x_next;
    y = y_next;
    z = z_next;
end

xout = x;
yout = y;
end
```

A.2 CORDIC.m

```

%% CORDIC ALGORITHM - FIXED TONES AND REPEATED LFM SWEEP
% This script uses mcordic_r2.m to generate cosine/sine pairs without
% calling sin() or cos() inside the sample-generation loops.

clear;
clc;
close all;

%% General parameters
fs = 44e3; % Sampling frequency [Hz]
steps = 32; % CORDIC iterations (approximately 32-bit precision)
angle_table = atan(2.^(-(0:steps-1)));

% CORDIC gain for the selected number of iterations.
cordic_gain = prod(sqrt(1 + 2.^(-2*(0:steps-1))));

% Fixed input vector. The desired output amplitude is 0.5.
xin = 0.5;
yin = 0;

% Simulated 1T31-style input preparation. The value is first truncated to
% 21 fractional bits and then pre-scaled by the inverse CORDIC gain.
x_quantized = (floor(xin * 2^21) * 2^10) / 2^31;
y_quantized = (floor(yin * 2^21) * 2^10) / 2^31;
x_actual = x_quantized / cordic_gain;
y_actual = y_quantized / cordic_gain;

fprintf('CORDIC steps: %d\n', steps);
fprintf('CORDIC gain: %.12f\n', cordic_gain);
fprintf('Pre-scaled initial x: %.12f\n\n', x_actual);

% Folder used by saveas().
output_folder = 'results_figures';
if ~exist(output_folder, 'dir')
    mkdir(output_folder);
end

%% PART 1 - Fixed tones: 440 Hz, 1 kHz, and 2.3 kHz
tone_frequencies = [440, 1000, 2300];
tone_duration = 1; % Seconds
N_tone = round(tone_duration * fs);
time_tone = (0:N_tone-1) / fs;
number_of_tones = length(tone_frequencies);

x_tones = zeros(number_of_tones, N_tone); % Cosine component
y_tones = zeros(number_of_tones, N_tone); % Sine component
tone_spectra = zeros(number_of_tones, floor(N_tone/2));
frequency_axis_tone = (0:floor(N_tone/2)-1) * (fs / N_tone);

for tone_index = 1:number_of_tones
    frequency = tone_frequencies(tone_index);
    phase = 0;
    phase_increment = 2 * pi * frequency / fs;

    for n = 1:N_tone
        [x_rotated, y_rotated] = mcordic_r2( ...
            x_actual, y_actual, angle_table, phase);
    end
end

```

CORDIC.m continued (2/5)

A.2 CORDIC.m (continued)

```

        x_tones(tone_index, n) = x_rotated;
        y_tones(tone_index, n) = y_rotated;

        % Phase accumulator update and wrapping to [-pi, pi].
        phase = phase + phase_increment;
        if phase > pi
            phase = phase - 2*pi;
        elseif phase < -pi
            phase = phase + 2*pi;
        end
    end

    complex_tone = x_tones(tone_index, :) + 1i*y_tones(tone_index, :);
    tone_fft = fft(complex_tone) / N_tone;
    tone_power = abs(tone_fft).^2;
    tone_spectra(tone_index, :) = tone_power(1:floor(N_tone/2));

    [~, peak_bin] = max(tone_spectra(tone_index, :));
    measured_frequency = frequency_axis_tone(peak_bin);
    fprintf('Tone %4d Hz -> FFT peak at %.1f Hz\n', frequency, measured_frequency);
end

% Time-domain plots (first 5 ms).
visible_samples = round(0.005 * fs);
figure(1);
for tone_index = 1:number_of_tones
    subplot(number_of_tones, 2, 2*tone_index-1);
    plot(time_tone(1:visible_samples), ...
        x_tones(tone_index, 1:visible_samples), 'LineWidth', 1.1);
    grid on;
    title(sprintf('%g Hz - cosine output', tone_frequencies(tone_index)));
    xlabel('Time (s)');
    ylabel('Amplitude');
    ylim([-0.55, 0.55]);

    subplot(number_of_tones, 2, 2*tone_index);
    plot(time_tone(1:visible_samples), ...
        y_tones(tone_index, 1:visible_samples), 'LineWidth', 1.1);
    grid on;
    title(sprintf('%g Hz - sine output', tone_frequencies(tone_index)));
    xlabel('Time (s)');
    ylabel('Amplitude');
    ylim([-0.55, 0.55]);
end
sgtitle('CORDIC-generated fixed tones');
saveas(gcf, fullfile(output_folder, '01_fixed_tones_time.png'));

% Frequency-domain plots.
figure(2);
for tone_index = 1:number_of_tones
    subplot(number_of_tones, 1, tone_index);
    plot(frequency_axis_tone, 10*log10(tone_spectra(tone_index, :) + eps), ...
        'LineWidth', 1.1);
    grid on;
    xlim([0, 4000]);
    title(sprintf('CORDIC spectrum - %g Hz tone', tone_frequencies(tone_index)));
    xlabel('Frequency (Hz)');
    ylabel('Power (dB)');
end

```

CORDIC.m continued (3/5)

A.2 CORDIC.m (continued)

```

end
saveas(gcf, fullfile(output_folder, '02_fixed_tones_spectrum.png'));

%% PART 2 - Repeated LFM sweep from 500 Hz to 3 kHz
f_start = 500;
f_end = 3000;
lfm_duration = 2; % Total duration [s]
ramp_duration = 1; % Duration of each repeated ramp [s]
N_lfm = round(lfm_duration * fs);
samples_per_ramp = round(ramp_duration * fs);
frequency_increment = (f_end - f_start) / (samples_per_ramp - 1);

phase = 0;
x_lfm = zeros(1, N_lfm);
y_lfm = zeros(1, N_lfm);
instantaneous_frequency = zeros(1, N_lfm);

for k = 1:N_lfm
    ramp_sample = mod(k - 1, samples_per_ramp);
    current_frequency = f_start + ramp_sample * frequency_increment;
    instantaneous_frequency(k) = current_frequency;

    phase_increment = 2 * pi * current_frequency / fs;
    phase = phase + phase_increment;

    if phase > pi
        phase = phase - 2*pi;
    elseif phase < -pi
        phase = phase + 2*pi;
    end

    [x_rotated, y_rotated] = mcordic_r2( ...
        x_actual, y_actual, angle_table, phase);
    x_lfm(k) = x_rotated;
    y_lfm(k) = y_rotated;
end

time_lfm = (0:N_lfm-1) / fs;
complex_lfm = x_lfm + 1i*y_lfm;

% Time-domain detail around the ramp reset at t = 1 s.
figure(3);
plot(time_lfm, y_lfm, 'LineWidth', 1.0);
grid on;
xlim([0.995, 1.005]);
ylim([-0.55, 0.55]);
title('CORDIC LFM signal around the 1 s ramp reset');
xlabel('Time (s)');
ylabel('Sine component');
saveas(gcf, fullfile(output_folder, '03_lfm_time_detail.png'));

% Overall LFM spectrum.
lfm_fft = fft(complex_lfm) / N_lfm;
lfm_power = abs(lfm_fft).^2;
frequency_axis_lfm = (0:N_lfm-1) * (fs / N_lfm);

figure(4);
plot(frequency_axis_lfm(1:floor(N_lfm/2)), ...

```

CORDIC.m continued (4/5)

A.2 CORDIC.m (continued)

```

    10*log10(lfm_power(1:floor(N_lfm/2)) + eps), 'LineWidth', 1.0);
grid on;
xlim([0, 5000]);
title('Spectrum of the repeated CORDIC LFM waveform');
xlabel('Frequency (Hz)');
ylabel('Power (dB)');
saveas(gcf, fullfile(output_folder, '04_lfm_spectrum.png'));

% Toolbox-independent short-time Fourier transform.
window_length = 1024;
overlap = 512;
hop = window_length - overlap;
nfft = 1024;
number_of_frames = floor((N_lfm - window_length) / hop) + 1;

hann_window = 0.5 - 0.5*cos(2*pi*(0:window_length-1)/(window_length-1));
stft_matrix = zeros(nfft/2 + 1, number_of_frames);

for frame_index = 1:number_of_frames
    first_sample = (frame_index - 1) * hop + 1;
    frame_samples = first_sample:first_sample + window_length - 1;
    windowed_frame = complex_lfm(frame_samples).' .* hann_window;
    frame_fft = fft(windowed_frame, nfft);
    stft_matrix(:, frame_index) = frame_fft(1:nfft/2 + 1);
end

stft_frequency = (0:nfft/2) * fs / nfft;
stft_time = ((0:number_of_frames-1) * hop + window_length/2) / fs;

figure(5);
imagesc(stft_time, stft_frequency, 20*log10(abs(stft_matrix) + eps));
axis xy;
ylim([0, 5000]);
colorbar;
title('Spectrogram of the repeated CORDIC LFM waveform');
xlabel('Time (s)');
ylabel('Frequency (Hz)');
saveas(gcf, fullfile(output_folder, '05_lfm_spectrogram.png'));

%% PART 3 - Numerical validation against MATLAB sin/cos
validation_angles = linspace(-pi, pi, 2001);
x_validation = zeros(size(validation_angles));
y_validation = zeros(size(validation_angles));

for index = 1:length(validation_angles)
    [x_validation(index), y_validation(index)] = mcordic_r2( ...
        x_actual, y_actual, angle_table, validation_angles(index));
end

x_reference = xin * cos(validation_angles);
y_reference = xin * sin(validation_angles);
vector_error = sqrt((x_validation - x_reference).^2 + ...
    (y_validation - y_reference).^2);

fprintf('\nMaximum vector error over [-pi, pi]: %.3e\n', max(vector_error));
fprintf('RMS vector error over [-pi, pi]: %.3e\n', ...
    sqrt(mean(vector_error.^2)));

```

CORDIC.m continued (5/5)

A.2 CORDIC.m (continued)

```
figure(6);
semilogy(validation_angles, vector_error + eps, 'Linewidth', 1.1);
grid on;
title('CORDIC vector error with 32 iterations');
xlabel('Angle (rad)');
ylabel('Euclidean error');
saveas(gcf, fullfile(output_folder, '06_cordic_error.png'));

%% Save generated data for later analysis
save('cordic_results.mat', ...
    'fs', 'steps', 'cordic_gain', 'tone_frequencies', ...
    'time_tone', 'x_tones', 'y_tones', 'tone_spectra', ...
    'frequency_axis_tone', 'time_lfm', 'x_lfm', 'y_lfm', ...
    'instantaneous_frequency', 'frequency_axis_lfm', 'lfm_power', ...
    'validation_angles', 'vector_error');

fprintf('\nFigures were saved in the folder: %s\n', output_folder);
fprintf('Numerical results were saved in cordic_results.mat\n');
```

Appendix B. Execution instructions

4. Place CORDIC.m and mcordic_r2.m in the same MATLAB folder.
5. Select that folder as the MATLAB Current Folder.
6. Run CORDIC.m from the Command Window or editor.
7. Inspect the figures saved in results_figures and the variables stored in cordic_results.mat.
8. The script is self-contained and does not require the Signal Processing Toolbox.

End of report